

```

import numpy as np
import matplotlib.pyplot as plt

a0=100 #m
b0=52 #m

a1=18 #m
b1=185 #m

r1=25 #m
r2=72 #m

#Formulation du problème
theta0=0*np.pi/180

def f(a):
    """fonction définissant la loi x/y de la table à bille
    a: angle table
    theta : angle moteur"""
    return (12**2 - (r1**2*cos(theta0-b0*np.cos(a)+a0*np.sin(a)+a0**2))

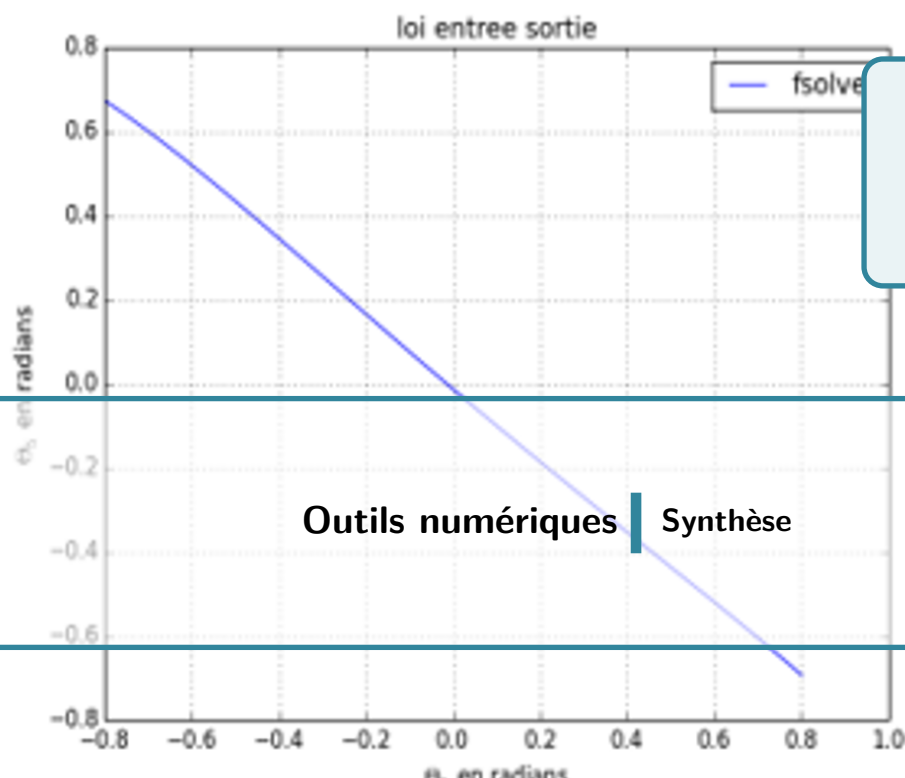
#Trace de la courbe f(alpha) pour alpha variant de -30° à 30°
a_min=-18*np.pi/180
a_max=18*np.pi/180
ls_alpha=np.linspace(a_min,a_max,100)
lf=[]
for alpha in ls_alpha:
    lf.append(f(alpha))

plt.figure(figsize=(10,6))
plt.plot(ls_alpha,lf)

plt.grid(True)
#On observe que la courbe coupe l'axe des abscisses par alpha env 0

#Résolution de f(alpha)=0 par dichotomie
def dichotomie(f, a, b, epsilon):
    """Résolution de l'équation f(x)=0 par la méthode de dichotomie"""
    while (b-a) > epsilon:
        m = (a+b)/2
        if f(m) < 0:
            a = m
        else:
            b = m
    return (a+b)/2

```



PTSI

Cycle NUM

Outils numériques | Synthèse

## Chapitre 2

### Intégration et dérivation numériques

#### Cours

##### Savoirs et compétences :

- Résoudre numériquement une équation ou un système d'équations (réécriture des équations d'un problème, résolution de problèmes du type  $f(x)=0$ , intégration et dérivation numérique).

|     |                                                               |   |
|-----|---------------------------------------------------------------|---|
| 1   | Principe quand la fonction est connue sur l'intervalle étudié | 2 |
| 2   | Méthode des rectangles                                        | 3 |
| 2.1 | Présentation                                                  | 3 |
| 2.2 | Ordre de la méthode numérique                                 | 4 |
| 2.3 | Convergence, erreur commise                                   | 4 |
| 2.4 | Algorithme et complexité                                      | 5 |
| 3   | Méthode des trapèzes                                          | 5 |
| 3.1 | Présentation                                                  | 5 |
| 3.2 | Ordre, convergence/erreur                                     | 6 |
| 3.3 | Algorithme et complexité                                      | 6 |
| 4   | A partir de valeurs mesurées                                  | 6 |
| 4.1 | Implémentation en python                                      | 7 |
| 5   | Utilisation d'une bibliothèque de python                      | 7 |
| 5.1 | Scipy : quad                                                  | 7 |
| 5.2 | Scipy : cumtrapz                                              | 7 |
| 5.3 | Scipy : trapez                                                | 8 |
| 6   | La dérivation numérique                                       | 9 |
| 6.1 | Principe                                                      | 9 |
| 6.2 | A partir de données sous forme de tableau                     | 9 |
| 6.3 | Scipy                                                         | 9 |
| 6.4 | Numpy                                                         | 9 |

Dans ce chapitre, on va décrire deux méthodes numériques classiques d'intégration (ou "de quadrature") permettant d'obtenir une valeur approchée d'une intégrale sur un segment : la méthode des rectangles et la méthode des trapèzes. Puis nous traiterons la dérivation numérique.

## 1 Principe quand la fonction est connue sur l'intervalle étudié

Soient  $a$  et  $b$  deux réels tels que  $a < b$ .

Soit  $f : [a, b] \rightarrow \mathbb{R}$  une fonction continue sur le segment  $[a, b]$ . On cherche une valeur approchée de

$$\int_a^b f(x) dx$$

On choisit, pour commencer, une subdivision  $\sigma = (a_0, \dots, a_n)$  de  $[a, b]$  : c'est un  $(n+1)$ -uplet de points de  $[a, b]$  qui vérifient :

$$a_0 = a < a_1 < \dots < a_n = b$$

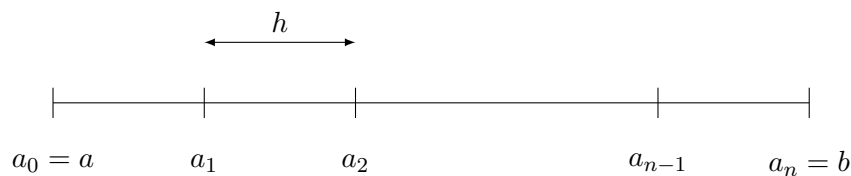
Par la relation de Chasles,

$$\int_a^b f(x) dx = \sum_{i=0}^{n-1} \int_{a_i}^{a_{i+1}} f(x) dx$$

On est donc ramené à évaluer l'intégrale de  $f$  sur un « petit » segment  $[a_i, a_{i+1}]$ .

On va remplacer  $f$  sur chaque segment  $[a_i, a_{i+1}]$  par une fonction dont on sait calculer l'intégrale.

Dans ce chapitre, on travaillera avec une subdivision de points uniformément distribués c'est-à-dire avec un pas constant  $h = \frac{b-a}{n}$



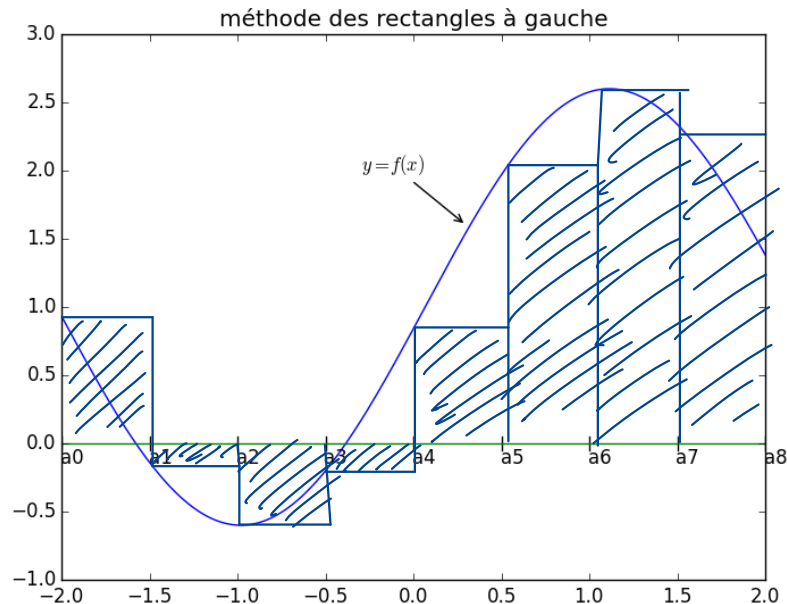
On a alors :  $\forall i \in \{0, \dots, n\}, a_i = a + ih = a + i \frac{b-a}{n}$

## 2 Méthode des rectangles

### 2.1 Présentation

*Méthode des rectangles à gauche*

Soit  $i \in \{0, \dots, n-1\}$ . On remplace  $f$  sur le segment  $[a_i, a_{i+1}]$  par la fonction constante égale à  $f(a_i)$ . Faisons apparaître ce procédé sur le schéma suivant :



La valeur de

$$\int_{a_i}^{a_{i+1}} f(x) dx$$

est proche de l'aire du rectangle de base  $[a_i, a_{i+1}]$  et de hauteur  $f(a_i)$ , qui vaut :  $\frac{b-a}{n} f(a_i)$ .

L'intégrale de  $f$  sur  $[a, b]$  entier est alors approchée par la somme  $R_n(f)$  des aires algébriques de ces rectangles de base  $[a_i, a_{i+1}]$ .

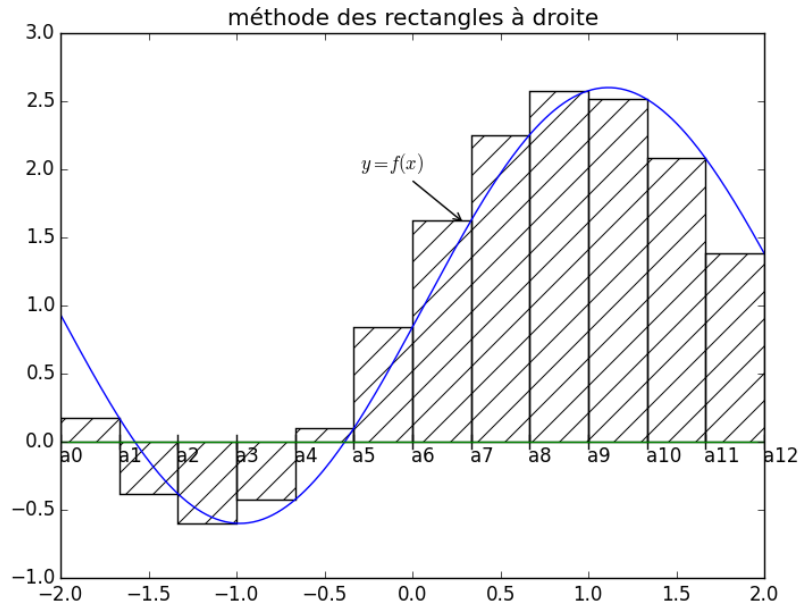
Calculons  $R_n(f)$  :

$$\begin{aligned} R_n(f) &= \sum_{i=0}^{n-1} \frac{b-a}{n} f(a_i) \\ &= \underbrace{\frac{b-a}{n}}_h \sum_{i=0}^{n-1} f(a_i) \end{aligned}$$

### Méthode des rectangles à droite

Dans la version précédente, sur le segment  $[a_i, a_{i+1}]$ , on a choisi la valeur constante "de gauche",  $f(a_i)$ . Il existe aussi une méthode des rectangles à droite : sur le segment  $[a_i, a_{i+1}]$ , on remplace  $f$  par la fonction constante égale à  $f(a_{i+1})$ .

Cela donne le schéma suivant :



On note  $R'_n(f)$  la somme des aires algébriques des rectangles qui va approcher l'intégrale. On a :

$$R'_n(f) = \frac{b-a}{n} \sum_{i=1}^n f(a_i)$$

$$\text{Ainsi, } R_n(f) - R'_n(f) = \frac{b-a}{n} (f(a_0) - f(a_n))$$

## 2.2 Ordre de la méthode numérique

**Définition :** On dit qu'une méthode d'intégration numérique est *d'ordre n* si la formule approchée est exacte pour tout polynôme de  $R_n[X]$  (espace vectoriel des polynômes de degré  $\leq n$ ) et inexacte pour au moins un polynôme de  $R_{n+1}[X]$ .

La méthode des rectangles (à gauche et à droite) est d'ordre 0.

## 2.3 Convergence, erreur commise

Si  $f$  est de classe  $C^1$  sur  $[a, b]$  alors on peut démontrer que :

$$\exists M \in \mathbb{R}_+, \forall n \in \mathbb{N}^*, \left| \int_a^b f(x) dx - R_n(f) \right| \leq \frac{M}{n},$$

ce qui se résume en :  $\int_a^b f(x) dx - R_n(f) = O\left(\frac{1}{n}\right)$

On a un résultat analogue pour la méthode des rectangles à droite.

## 2.4 Algorithme et complexité

### *Algorithme de la méthode des rectangles à gauche*

Écrivons en python une fonction `rectangle_gauche` d'arguments une fonction `f`, des flottants `a` et `b` et le nombre `n` de subdivisions. Cette fonction renvoie une valeur approchée de l'intégrale de  $f$  sur le segment  $[a, b]$  par la méthode des rectangles à gauche.

```
def rectangle_gauche(f, a, b, n):
    s = 0
    h = (b - a) / n
    for i in range(n):
        # Invariant: s contient f(a0)+...+f(ai-1) si i>=1 et 0 si i=0
        s = s + f(a + i * h)
        # s contient f(a0)+...+f(ai)
        # pour i = n - 1, s contient f(a0)+...+f(an-1)
    return (s * h)
```

### *Complexité*

La taille du problème est l'entier  $n$ . Nous allons compter le nombre d'affectations.

- ★ Il y a 2 affectations initiales
- ★ Pour  $i$  fixé entre 0 et  $n - 1$ , on effectue 1 affectations.

Au total,  $2 + n$  affectations donc une complexité en  $O(n)$ .

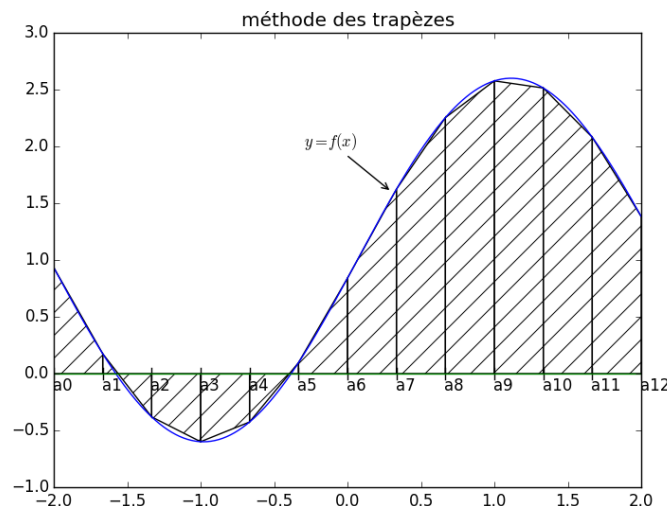
### *Algorithme de la méthode des rectangles à droite*

Pour programmer la méthode des rectangles à droite, on modifie l'algorithme précédent en utilisant une boucle `for i in range(1, n + 1)`.

## 3 Méthode des trapèzes

### 3.1 Présentation

Sur chaque segment  $[a_i, a_{i+1}]$ , on approche la fonction  $f$  par la fonction affine qui coïncide avec  $f$  en  $a_i$  et  $a_{i+1}$ .



Le trapèze de base  $[a_i, a_{i+1}]$  a pour aire :  $\frac{b-a}{2n}(f(a_i) + f(a_{i+1}))$ .

L'intégrale est approchée par la somme des aires algébriques  $T_n(f)$  des trapèzes de base  $[a_i, a_{i+1}]$ . Ainsi,

$$T_n(f) = \frac{b-a}{2n} \sum_{i=0}^{n-1} (f(a_i) + f(a_{i+1}))$$

Remarque :

On peut noter que  $T_n(f) = \frac{h}{2} \left( \sum_{i=0}^{n-1} f(a_i) + \sum_{i=0}^{n-1} f(a_{i+1}) \right) = \frac{h}{2} \left( \sum_{i=0}^{n-1} f(a_i) + \sum_{i=1}^n f(a_i) \right)$ .

$T_n(f)$  est la moyenne de  $R_n(f)$  et de  $R'_n(f)$ .

### 3.2 Ordre, convergence/erreur

Cette méthode est d'ordre 1

Si  $f$  est de classe  $C^2$  sur  $[a, b]$  alors on peut démontrer que :  $\int_a^b f(x) dx - T_n(f) = O\left(\frac{1}{n^2}\right)$

### 3.3 Algorithme et complexité

Écrivons en python une fonction `trapeze` d'arguments une fonction `f`, des flottants `a` et `b` et le nombre de subdivision `n`. Cette fonction renvoie une valeur approchée de l'intégrale de  $f$  sur le segment  $[a, b]$  par la méthode des trapèzes.

```
def trapeze(f, a, b, n):
    h = (b - a) / n
    s = 0
    for i in range(n):
        s = s + f(a + i * h) + f(a + (i + 1) * h)
    return (s * h / 2)
```

Remarque : Au lieu de renvoyer une valeur, on peut choisir de renvoyer une liste des valeurs évaluées pour chaque pas de calcul `i`.

C'est, comme pour la méthode des rectangles, une complexité en  $O(n)$  donc une complexité linéaire.

## 4 A partir de valeurs mesurées

Dans cette partie, la fonction  $f$  n'est ni connue ni définie sur l'intervalle considéré  $[a, b]$ .

L'évaluation de l'intégrale se fait à partir de deux listes de valeurs mesurées.

Ces valeurs sont obtenues par l'exploitation des mesures d'un capteur sous forme d'un fichier `csv` ou `txt`.

On extrait de ce fichier deux listes ou deux tableaux de dimension 1. Une liste stocke les temps (les instants de mesure) et l'autre liste stocke la grandeur mesurée.

## 4.1 Implémentation en python

```
def integraleRectangleAGauche(LX:list,LY:list)->list,list:
    '''évalue a chaque pas de calcul, la valeur approchée de l'intégrale
    par la méthode des rectangles à gauche et stocke cette valeur dans une liste
    LX la liste des temps
    LY la liste de la grandeur mesurée'''
    # initialisation
    LX_int=[0] # l'instant initial est considéré a 0
    LY_int=[0] # l'évaluation de l'intégrale a l'instant 0 est considéré nulle
    # si ces valeurs ne sont pas nulles initialement, on pourra réaliser une
    # translation par la suite
    t=0
    s=0
    for i in range(len(LX)-1):
        h=LX[i+1]-LX[i]
        t+=h
        s+=LY[i]*h
        LY_int.append(s)
        LX_int.append(t)
    return (LX_int,LY_int)
```

## 5 Utilisation d'une bibliothèque de python

### 5.1 Scipy : quad

On peut utiliser la commande quad de la sous-bibliothèque `scipy.integrate`.

Ici, on l'utilise pour calculer  $\int_0^1 x^2 dx$ .

```
import scipy.integrate as spi
def f(x):
    return(x ** 2)

valeur, erreur = spi.quad(f, 0, 1)
print (valeur, erreur)
(0.3333333333333337, 3.700743415417189e-15)
```

La réponse est un tuple formée d'une valeur approchée de l'intégrale et d'une estimation de l'erreur commise.

### 5.2 Scipy : cumtrapz

Intégration d'une liste de valeurs, l'intégration cumulée permet d'avoir une liste de valeurs contenant le résultat de l'intégrale à chaque instant (sous forme de tableau).

```
from scipy.integrate import cumtrapz # module d'intégration numérique
import numpy as np
import matplotlib.pyplot as plt

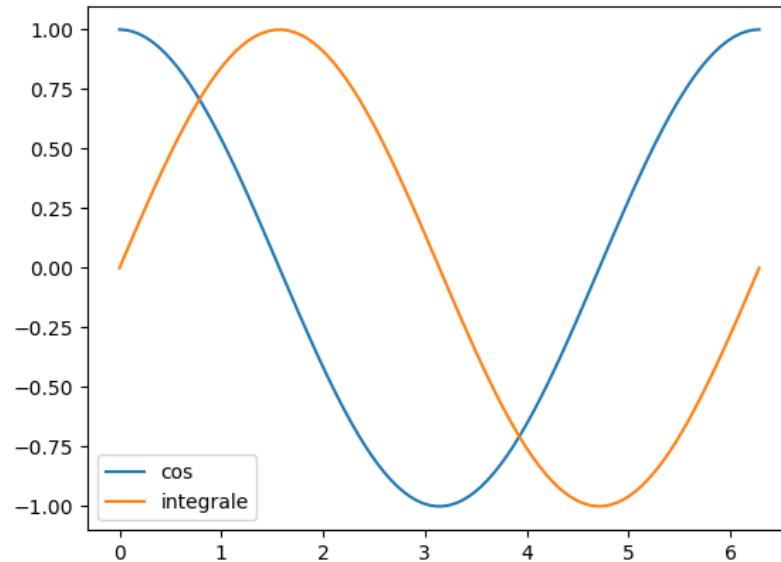
def f(x):
    return np.cos(x)

x=np.linspace(0,2*np.pi,100)
y=f(x)

vect=cumtrapz(y,x,initial=0)

plt.figure()
plt.plot(x,y,label='cos')
plt.plot(x,vect,label='integrale')
```

```
plt.legend()
plt.show()
```



### 5.3 Scipy : trapz

La fonction `trapeze` est une fonction prédéfinie dans du module `scipy.integrate` qui prend pour arguments deux listes :

```
trapz(y, x=None, dx=1.0, axis=-1)
    Integrate along the given axis using the composite trapezoidal rule.
    Integrate `y` (`x`) along given axis.
    Parameters
    -----
    y : array_like Input array to integrate.
    x : array_like, optional If `x` is None, then spacing between all `y` elements is `dx`.
    dx : scalar, optional If `x` is None, spacing given by `dx` is assumed. Default is 1.
    axis : int, optional Specify the axis.

    Returns
    -----
    trapz : float Definite integral as approximated by trapezoidal rule.
```

Cette fonction `trapeze` existe aussi dans la bibliothèque `numpy` : `trapz(y,x)`.

## 6 La dérivation numérique

### 6.1 Principe

On utilise le taux d'accroissement vu en mathématique,

$$f'(x_0) \approx \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$

Soit pour une discrétisation de la fonction par pas  $h$  avec  $h = x_{i+1} - x_i$  :

$$f'\left(\frac{x_{i+1} + x_i}{2}\right) \approx \frac{f(x_{i+1}) - f(x_i)}{h}$$

La pente est calculée entre deux abscisses, on peut associer la valeur de la dérivée à l'abscisse milieu ou "centrée". L'avantage de cette méthode est qu'elle respecte une certaine symétrie entre les abscisses qui permettent le calcul et la position située au centre à laquelle on associe la valeur de la dérivée.

### 6.2 A partir de données sous forme de tableau

On construit le tableau des valeurs dérivées suivant la relation :

$$yp_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

On conserve ici la liste des abscisses initiale sans calculer les valeurs centrées. Pour un pas suffisamment petit, on ne voit pas de différence avec la méthode précédente.

```
# Fonction qui calcul la dérivée de deux listes
def derivee(y, x):
    d = []
    for i in range(0, len(y)-1):
        dy = y[i+1]-y[i]
        dx = x[i+1]-x[i]
        d.append(dy/dx)
    return d
```

### 6.3 Scipy

On utilisera la fonction `derivative(f,x0,dx)` du module `scipy.misc` pour calculer la dérivée de  $f$  en  $x_0$ .

```
from scipy.misc import derivative # Fonction de dérivation

# Calcul de la dérivée
# dx correspond à l'intervalle entre deux points pour le calcul
res = derivative(function, x0, dx=1e-6)
```

Si  $x$  est un tableau `numpy` de valeurs alors `derivative(f,x,dx)` renvoie un tableau des valeurs de la fonction dérivée en ces points.

```
# Points où l'on veut la dérivée
x = np.linspace(-10, 10, 100)

# Calcul de la dérivée
res = derivative(function, x, dx=1e-6)
```

### 6.4 Numpy

Pour dériver une liste de points, on utilise la fonction `diff(x)` du module `numpy` qui calcule la différence entre deux points successifs et on divise le tout par le pas. **le pas doit être constant.**

```
import numpy as np
# x est un tableau numpy
pas = 0.04
v_calc = np.diff(x)/pas
```